Reporte

| | |
|---|---|
| **Nombres**: Equipo 11<br>Christopher Cortez Garcia<br>Oscar Elias Vera Moreno<br>Christofer Aaron Torres Puga | **Matrículas**:<br>2159313<br>2160859<br>2159639 |
| **Nombre del curso:** Componentes de Lógica de Negocio | **Nombre del profesor**: Miguel Ángel Gómez Marroquín |
| **Modulo Netacad**: 3 | **Actividad**: Diseño Arquitectural del sistema (Fitzone) |
| **Fecha**: 17/10/2026 | |
| **Bibliografía (Formato APA)**<br>Documento de caso asignado enviado por el profesor. (2026). Caso 11: FitZone. | |

## Propósito de la actividad

El objetivo es organizar las reglas y responsabilidades del sistema antes de programarlo por completo. Este documento demuestra cómo el equipo pasó de los requerimientos de la Unidad 2 a una arquitectura que separa dominio, aplicación e infraestructura para el proceso "Reservación de clase en FitZone".

La actividad no consistió en pedirle a una IA una solución terminada: en cada etapa se solicitaron alternativas, se contrastaron contra el caso real y se documentó por qué una propuesta corresponde o no al negocio (ver bitácora de IA, sección 14).

## Introducción

La transformación de un documento de requerimientos de negocio y casos de uso en una arquitectura de software robusta, escalable y mantenible exige un análisis riguroso de las reglas, restricciones y comportamientos esperados del sistema. El presente informe detalla la conceptualización arquitectural inicial para el ecosistema tecnológico de FitZone, cuyas operaciones centrales abarcan la gestión de membresías, el control automatizado de accesos físicos y la reservación dinámica de clases.
Para abordar este desafío ingenieril, el desarrollo del diseño se ha estructurado siguiendo estrictamente las tres primeras fases de la definición arquitectural orientada al dominio estipuladas en la rúbrica de evaluación.

## Etapa 1: Preparar el caso

El establecimiento de una arquitectura de software coherente exige delimitar los flujos transaccionales que poseen el mayor impacto estratégico. Para FitZone, el objetivo primario

es doble: mitigar la fuga de capacidad operativa por inasistencias ("no-shows") y automatizar el control de acceso.

Se han seleccionado cinco casos de uso arquitecturalmente significativos que abarcan flujos normales, flujos alternativos basados en ventanas temporales asíncronas y flujos de excepción.

| ID | Caso de uso | Actor | Objetivo / Entrada | Reglas relacionadas (evidencia U2) | Resultado esperado |
|---|---|---|---|---|---|
| CU-01 | Reservar clase en línea (flujo normal) | Socio | Asegurar su participación · socioId, claseId | N2, US-001, US-002, US-013 (límite por plan, vigencia activa, cupo > 0) | Reserva confirmada; decremento atómico de 1 en el cupo disponible |
| CU-02 | Inscripción en lista de espera al agotarse el cupo (flujo alterno) | Socio | No perder la oportunidad de asistir · socioId, claseId | N3, US-003, US-005 (cupo == 0, orden FIFO, ventana de 30 min para confirmar) | Socio registrado en lista de espera con posición asignada, sin exceder la capacidad estructural |
| CU-03 | Cancelación anticipada sin penalización (flujo alterno) | Socio / Recepción | Liberar su lugar · reservaId | N2, N3, US-004, US-005 (cancelación con ≥3h de anticipación, reasignación FIFO) | Reserva liberada sin penalización, cupo incrementado y notificación automática al primer socio en espera |
| CU-04 | Validación de acceso en recepción y rechazo por vigencia (flujo excepción) | Recepción / Sistema | Autorizar o negar el ingreso · socioId | N1, US-006, US-007, US-018 (verificación automática, motivo explícito de rechazo) | Acceso denegado en torniquete, registro auditable con motivo "Membresía Vencida" y cancelación automática de reservas activas futuras |
| CU-05 | Sanción automática por acumulación de no-shows (flujo excepción) | Sistema / Instructor | Aplicar la política de inasistencias · reservaId, asistió | N4, US-011, US-015, US-017 (3 no-shows/mes = bloqueo de 7 días en reservas) | Socio marcado como "Sancionado" para reservas durante 7 días; el acceso físico al gimnasio permanece permitido |

## Etapa 2: Elección del Patrón de Lógica de Negocio

Para determinar el patrón más adecuado para el ecosistema tecnológico de FitZone, se ha elaborado la siguiente matriz comparativa evaluando cada opción a través del prisma de las exigencias del caso de estudio documentado en la Unidad 2.

| Criterio del caso FitZone | Transaction Script | Table Module | Domain Model | Evidencia de trazabilidad U2 |
|---|---|---|---|---|
| Complejidad funcional de las reglas | Inadecuado: forzaría duplicar lógica compleja (ventana de 30 min, límites concurrentes por plan) en múltiples scripts. | Parcial: centraliza reglas por tabla individual, pero dificulta severamente lógicas transversales que cruzan socios, clases, planes y pagos. | Óptimo: encapsula las políticas de sanciones y asignación de lugares dentro de entidades ricas, aislando la complejidad de la infraestructura. | E-01: BRD N1/N2 y US-001/US-013 establecen reglas estrictas de límite de reservas según plan y validación de vigencia. |
| Interacción transversal entre conceptos | Deficiente: requiere consultas ad-hoc para verificar planes, reservas previas y listas de espera por cada script. | Deficiente: fomenta dependencias circulares entre el módulo de Reservas y el módulo de Membresías. | Óptimo: facilita la colaboración de objetos en memoria (una ClasePrograma delega lugares liberados directamente a su ListaDeEspera autónoma). | E-02: BRD N3 y US-003/US-005 detallan la asignación FIFO estricta con ventana de confirmación inamovible. |
| Protección de estados e invariantes | Altamente peligroso: objetos anémicos; cualquier script externo podría saltarse una validación de bloqueo. | Regular: protege la fila a nivel de esquema SQL, pero no representa semánticamente el ciclo de vida del socio. | Óptimo: protege férreamente invariantes de negocio inquebrantables, como el aforo máximo o las condiciones de bloqueo. | E-03: BRD N4 y US-011/US-017 definen la sanción asíncrona y automática de 7 días exactos de bloqueo al acumular 3 inasistencias. |
| Mantenibilidad y pruebas (criterio adicional) | Bajo: alto riesgo de código enredado ante la introducción futura de nuevas membresías. | Medio: depende de un mapeo relacional directo que ralentiza la refactorización. | Alto: aísla el núcleo mediante dependencias invertidas; permite testear reglas punitivas y flujos críticos sin base de datos activa. | E-04: BRD sección 2 expone como problema central la fuga de capacidad por no-shows y accesos inconsistentes. |

## Ventajas, riesgos y consecuencias por alternativa

- **Transaction Script**. Ventaja: rapidez de desarrollo inicial. Riesgo: duplicación de reglas (cut-off, FIFO, bloqueo) entre scripts.

  Consecuencia: errores de inconsistencia entre el flujo de reserva y el de cancelación al crecer la política de no-shows.

- **Table Module**. Ventaja: más orden que scripts sueltos. Riesgo: reglas que cruzan ClasePrograma + ListaDeEspera + Reserva (liberación automática FIFO) no tienen un lugar natural.

  Consecuencia: la lógica de coordinación terminaría mezclada en la capa de aplicación.

- **Domain Model**. Ventaja: encapsula reglas e invariantes donde pertenecen. Riesgo: mayor curva de diseño inicial (requiere identificar Aggregates y límites de consistencia).

  Consecuencia: mayor inversión en el mapa de dominio (Etapa 3), a cambio de mayor solidez ante las excepciones ya identificadas.

## Decisión Argumentada

Con base en la rigurosa evaluación cualitativa presentada y respaldada por la evidencia del análisis previo, la decisión arquitectural central establece la adopción irrestricta del **Domain Model** como patrón rector de lógica de negocio.

La fundamentación de esta decisión radica profundamente en la naturaleza altamente condicional del dominio. Resolver la "fuga de capacidad" (BRD Sec 2) exige garantizar a nivel matemático que una sesión jamás sobrepase su capacidad máxima y que las cancelaciones detonen flujos asíncronos hacia la lista de espera (US-005). El patrón Transaction Script requeriría consultas secuenciales vulnerables a condiciones de carrera en alta concurrencia, permitiendo potencialmente la sobreventa. Por el contrario, el Domain Model permite que un objeto ClaseProgramada proteja su propio estado en memoria, procesando solicitudes atómicamente.

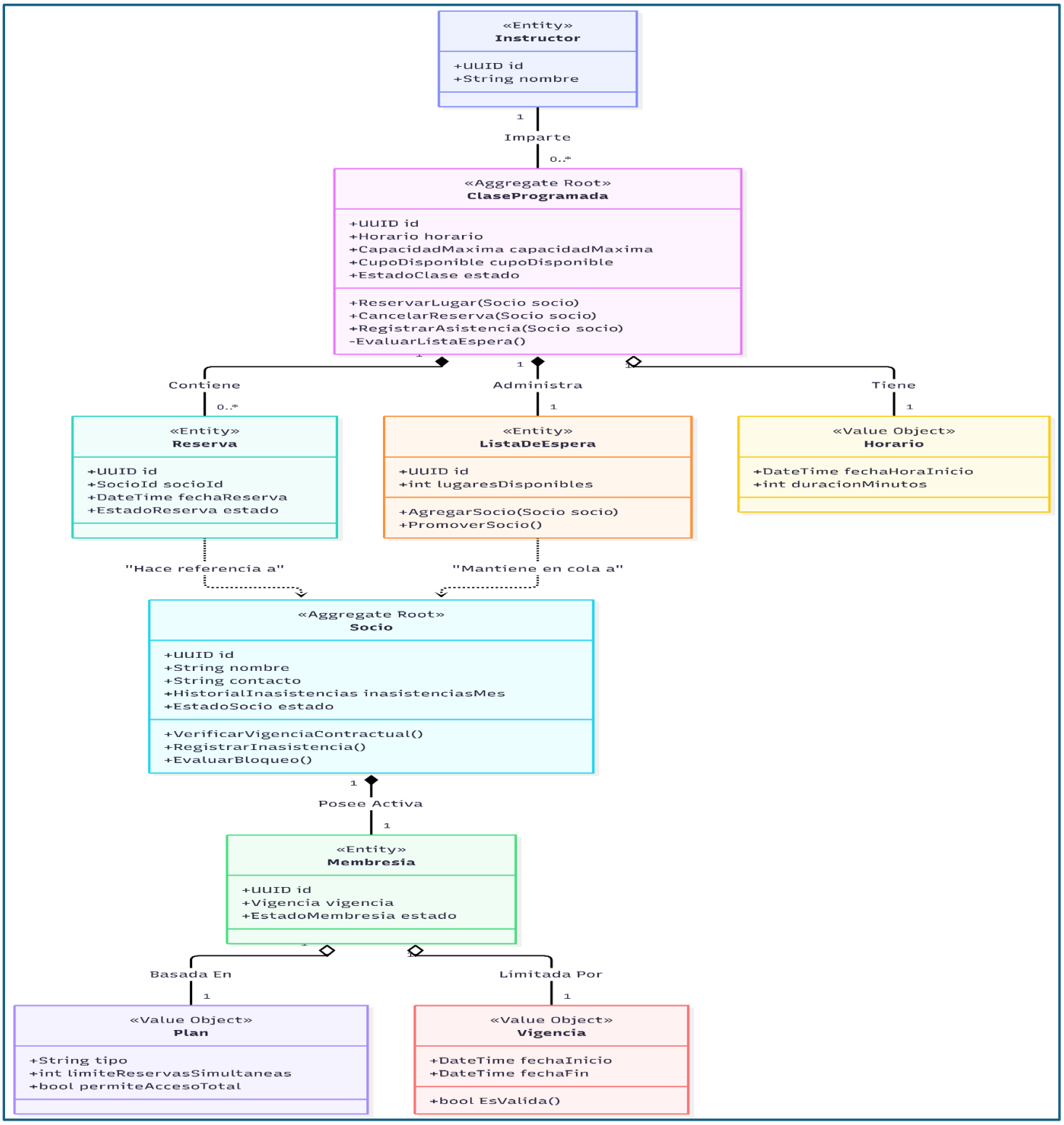
Asimismo, la regla de control de no-shows (US-017) evidencia por qué un modelo rico es imperativo. La penalización bloquea reservas por siete días pero salvaguarda el acceso físico. Un modelo Table Module requeriría constantes intercepciones procedimentales entre módulos; en cambio, en el Domain Model, la entidad Socio encapsula su estado, autoevalúa sus faltas y altera su ciclo de vida autónomamente, emitiendo eventos de dominio para notificar a otras partes del sistema.

## Etapa 3: Construcción del Mapa de Dominio

La justificación teórica del Domain Model requiere el uso extensivo del Diseño Orientado al Dominio (DDD). A continuación se delimitan las responsabilidades, los límites de consistencia y se provee el mapa visual del sistema.

| **Elemento** | **Respuesta del equipo para FitZone** |
|---|---|
| Lenguaje ubicuo | Socio, Membresía, Plan, ClasePrograma da, CupoDisponible, ListaDeEspera, No-show, Vigencia, Drop-in — mismos términos usados por el stakeholder, el glosario de negocio y el código. |
| Entidades | Socio, Membresía, ClasePrograma da, Instructor, Reserva — tienen identidad y ciclo de vida propio. |
| Value Objects | Vigencia (fechaInicio/fechaFin, inmutable: si se extiende, el objeto se destruye y se crea uno nuevo), Horario, CapacidadMaxima, CupoDisponible, Plan — definidos por sus valores, sin identidad propia. |
| Aggregates | ClasePrograma da (raíz): agrupa Reserva, ListaDeEspera, CapacidadMaxima y CupoDisponible. Socio (raíz): agrupa Membresía y Plan. |
| Domain Services | ServicioDeInscripcionExtraordinaria — lógica apátrida que orquesta el acceso "drop-in" (US-019) evaluando un pago externo sin modificar la membresía base del socio. |
| Domain Events | ReservaConfirmada, LugarLiberadoEnClase, ClaseCanceladaPorInstructor, SocioBloqueadoPorInasistencia, AccesoFisicoDenegado, MembresiaVencida — hechos históricos inmutables que interesan a otros componentes. |

## 3.1 Diagrama del Mapa de Dominio



## 3.2 Fichas de Agregados y Fronteras Transaccionales

| Aggregate | Raíz | Responsabilidades | Invariantes protegidas | Eventos emitidos |
|---|---|---|---|---|
| ClaseProgramada | ClaseProgramada (única interfaz para interactuar con reservas de la sesión) | Aceptar o rechazar reservas según cupo; coordinar liberaciones y reasignaciones FIFO de la lista de espera. | 1) Las reservas activas nunca superan numéricamente la CapacidadMaxima. 2) El orden de promoción en ListaDeEspera es estrictamente FIFO, sin excepciones comerciales. | ReservaConfirmada, LugarLiberadoEnClase, ClaseCanceladaPorInstructor |
| Socio | Socio | Autoevaluar elegibilidad para accesos físicos y reservas; monitorear cuotas de inasistencias. | 1) Un socio posee solo una Membresía activa a la vez. 2) Si el plan es 'Básico', las reservas simultáneas son ≤ 2. 3) Acumular 3 no-shows mensuales impone estado 'Bloqueado' para reservas (7 días). | SocioBloqueadoPorInasistencia, AccesoFisicoDenegado, MembresiaVencida |

## 3.3 Análisis de Decisiones Dudosas y Descartadas

**Decisión Descartada: Independizar la entidad Reserva como Agregado Raíz.**

- ***Planteamiento:*** Se debatió permitir que la aplicación insertara reservas directamente en la base de datos (estilo CRUD/Table Module) para mayor velocidad, en lugar de pasar siempre a través del objeto ClaseProgramada.
- ***Justificación del Rechazo:*** Hacer esto delegaría la responsabilidad de auditar el cupo físico a un servicio externo, generando condiciones de carrera bajo alta concurrencia. Al encapsular la Reserva dentro de ClaseProgramada, aceptamos cargar toda la clase en memoria RAM al reservar, pero garantizamos matemáticamente (Invariante) que es imposible sobreasignar espacios y que la lista de espera se active atómicamente.

**Decisión Descartada: Modelar "Saldos a Favor" para reembolsos por cancelaciones.**

- ***Planteamiento:*** Se consideró crear un Agregado Financiero para devolver dinero cuando un instructor cancela una clase (US-010).
- ***Justificación del Rechazo:*** Revisando los supuestos del BRD (Sección 1 y 6), las clases son un servicio incluido globalmente en la suscripción periódica. El dominio

no admite devoluciones unitarias. La arquitectura se simplificó implementando un Evento de Dominio (ClaseCanceladaPorInstructor) que la capa de aplicación escucha para notificar al usuario y ofrecer reagendamiento, sin contaminar el modelo con carteras virtuales innecesarias.

## Etapa 4: Diseñar la Service Layer

| Servicio | Método / caso de uso | Entrada | Colaboradores | Salida | Reglas que delega al dominio |
|---|---|---|---|---|---|
| ReservaService | ReservarClase (CU-01 / CU-02) | socioId, claseId | ISocioRepository, IClaseProgramadaRepository, Socio, ClaseProgramada | ReservaDTO (estado: Confirmada, ListaEspera, Rechazada) | Delega a Socio validar vigencia y bloqueos. Delega a ClaseProgramada validar aforo, registrar reserva o enviar a ListaDeEspera. |
| ControlAccesoService | ValidarAccesoFisico (CU-04) | socioId, torniqueteId | ISocioRepository, Socio | AccesoResultadoDTO (Permitido / Denegado + motivo) | Delega a Socio el método VerificarVigenciaContractual(). El servicio solo coordina la lectura y la respuesta al torniquete. |
| AsistenciaService | ProcesarInasistencia (CU-05) | socioId, claseId | ISocioRepository, IClaseProgramadaRepository, Socio | bool de éxito | Delega a Socio RegistrarInasistencia() y EvaluarBloqueo(). El dominio decide autónomamente si aplica la sanción de 7 días. |
| ReservaService | CancelarReserva (CU-03) | socioId, claseId | IClaseProgramadaRepository, ClaseProgramada, ListaDeEspera | CommandResult | Delega a ClaseProgramada liberar el lugar y promover al siguiente socio de la ListaDeEspera (orden FIFO). |
| DropInService | PagarAccesoExtraordinario (US-019) | socioId, claseId, monto | IClaseProgramadaRepository, ServicioDeInscripcionExtraordinaria | ReservaDTO (drop-in) | Delega al Domain Service ServicioDeInscripcionExtraordinaria validar cupo y procesar el pago sin tocar la Membresía del socio. |

## Revisión obligatoria

**¿El nombre del método expresa una acción del negocio?** Sí: ReservarClase, CancelarReserva, ValidarAccesoFisico, ProcesarInasistencia y PagarAccesoExtraordinario describen intenciones del usuario, no operaciones técnicas.

**¿El método coordina o contiene decisiones que deberían estar en el dominio?** Se revisó: ningún Application Service evalúa reglas directamente; todos delegan en el Aggregate correspondiente (ClaseProgramada.ReservarLugar(), Socio.EvaluarBloqueo(), Socio.VerificarVigenciaContractual()).

**¿El servicio depende de interfaces o de detalles concretos?** Depende solo de ISocioRepository e IClaseProgramadaRepository; no referencia SQL ni un ORM directamente.

**¿La operación tiene inicio y fin transaccional comprensible?** Sí: CancelarReserva es el caso más sensible porque toca dos operaciones del mismo Aggregate (liberar + promover) dentro de un único Unit of Work (ver Etapa 5).

**¿Los errores del dominio pueden explicarse en términos del negocio?** Sí: por ejemplo, ClaseProgramada rechaza una reserva con el motivo de negocio "CapacidadExcedida" en vez de una excepción técnica genérica.

## Etapa 5: Separar persistencia y transacciones

Aquí definimos los Repositorios guiados por tus *Aggregate Roots* (Socio y ClaseProgramada) para no depender de bases de datos relacionales en la lógica.

| Repository | Necesidad del dominio | Métodos propuestos | Objeto que recupera o guarda |
|---|---|---|---|
| ISocioRepository | Recuperar el estado actual del socio, su membresía y faltas, para validar si puede entrar al gimnasio o reservar. | ObtenerPorId(UUID id); Guardar(Socio socio) | Socio (Aggregate Root completo, con su Value Object Vigencia) |
| IClaseProgramadaRepository | Cargar la sesión con sus reservas actuales y lista de espera, para procesar de forma segura una nueva inscripción o cancelación. | ObtenerPorId(UUID id); Guardar(ClaseProgramada clase) | ClaseProgramada (Aggregate Root completo, incluye listas internas) |
| IInstructorRepository | Encontrar instructores disponibles para reasignar una clase sin instructor presente. | ConsultarDisponibles(Horario horario) | Instructor |

**Propuesta de Unit of Work:** El alcance transaccional debe asegurar la consistencia. Tomemos el caso **CU-03 (Cancelación anticipada)**. Cuando un usuario cancela, ocurren dos cosas:

1. Se libera su lugar en la ClaseProgramada.
2. Se promueve automáticamente al primer socio de la ListaDeEspera (flujo asíncrono FIFO). El **Unit of Work** iniciará al comenzar el Application Service y se confirmará (Commit) solo cuando el Aggregate de la Clase guarde el nuevo estado con el usuario promovido. Si hay un error al promover al nuevo socio o al guardar en la base de datos, se hace un Rollback completo para evitar que el lugar quede "volando" o se corrompa el aforo.

## Etapa 6: Separar Commands y Queries (CQRS básico)

Dado que FitZone requiere proteger estrictamente los aforos, pero a la vez permitir que cientos de usuarios consulten los horarios sin tirar el sistema, aplicar CQRS es muy útil.

| Tipo | Nombre | Datos de entrada | Handler | Resultado | Justificación |
|---|---|---|---|---|---|
| Command | CancelarReservaCommand | socioId, claseId | CancelarReservaHandler | CommandResult (Éxito/Error) | Modifica el estado del sistema (libera un lugar y mueve la lista de espera); no necesita devolver datos de la clase, solo confirmar si la acción se ejecutó. |
| Command | SancionarPorAusenciaCommand | socioId, claseId | SancionarPorAusenciaHandler | CommandResult | Cambia el estado del sistema sumando una falta; ejecuta lógica de dominio pura (invariante de bloqueo por 3 faltas). |
| Query | ConsultarDisponibilidadClasesQuery | fecha, sucursalId | ConsultarDisponibilidadHandler | List<ClaseDisponibleDTO> | Es solo lectura y de alto volumen (consulta masiva desde la app); no se necesita cargar el Aggregate ClaseProgramada completo, solo nombre, horario y cupo disponible. |
| Query | ConsultarEstadoSocioQuery | socioId | ConsultarEstadoSocioHandler | PerfilSocioDTO | Muestra al socio si su membresía está activa o si tiene el castigo de 7 días; lee de forma plana sin instanciar la lógica de validación de negocio. |

Parte del caso que no necesita esta separación: ValidarAccesoFisico (CU-04) es una consulta simple de un solo Aggregate sin reglas de escritura, por lo que se deja como método directo del ControlAccesoService en vez de crear un Query formal, el costo de un Handler adicional no se justifica para una sola lectura puntual en el torniquete.

## Etapa 7: Revisar la propuesta con SOLID

| Principio | Pregunta de revisión | Evidencia encontrada en FitZone |
|---|---|---|
| S | ¿Cada clase o servicio tiene una razón principal para cambiar? | ReservaService inicialmente mezclaba el envío de notificaciones con la coordinación de la cancelación; se separó la notificación a un NotificacionService aparte. |
| O | ¿Pueden agregar una variante de regla sin editar una cadena creciente de condicionales? | El umbral de no-shows (hoy 3) vive solo dentro de Socio.EvaluarBloqueo(); cambiarlo a 4 no tocaría AsistenciaService ni ClaseProgramada. |
| L | ¿Las implementaciones pueden sustituirse sin cambiar el comportamiento esperado? | Se evitó una jerarquía Reserva/ReservaDropIn por herencia; se usa un atributo esDropIn en |

| | | |
|---|---|---|
| | | Reserva para no romper el contrato con subtipos que rechacen operaciones. |
| I | ¿Cada consumidor depende solo de las operaciones que utiliza? | Se separó IClaseProgramadaRepository (consultas de disponibilidad y guardado) de ISocioRepository en vez de un repositorio genérico con todos los métodos. |
| D | ¿La aplicación y el dominio dependen de abstracciones? | Los Application Services dependen de ISocioRepository e IClaseProgramadaRepository, no de una implementación SQL o de un ORM concreto. |

## Hallazgos registrados (mínimo cinco)

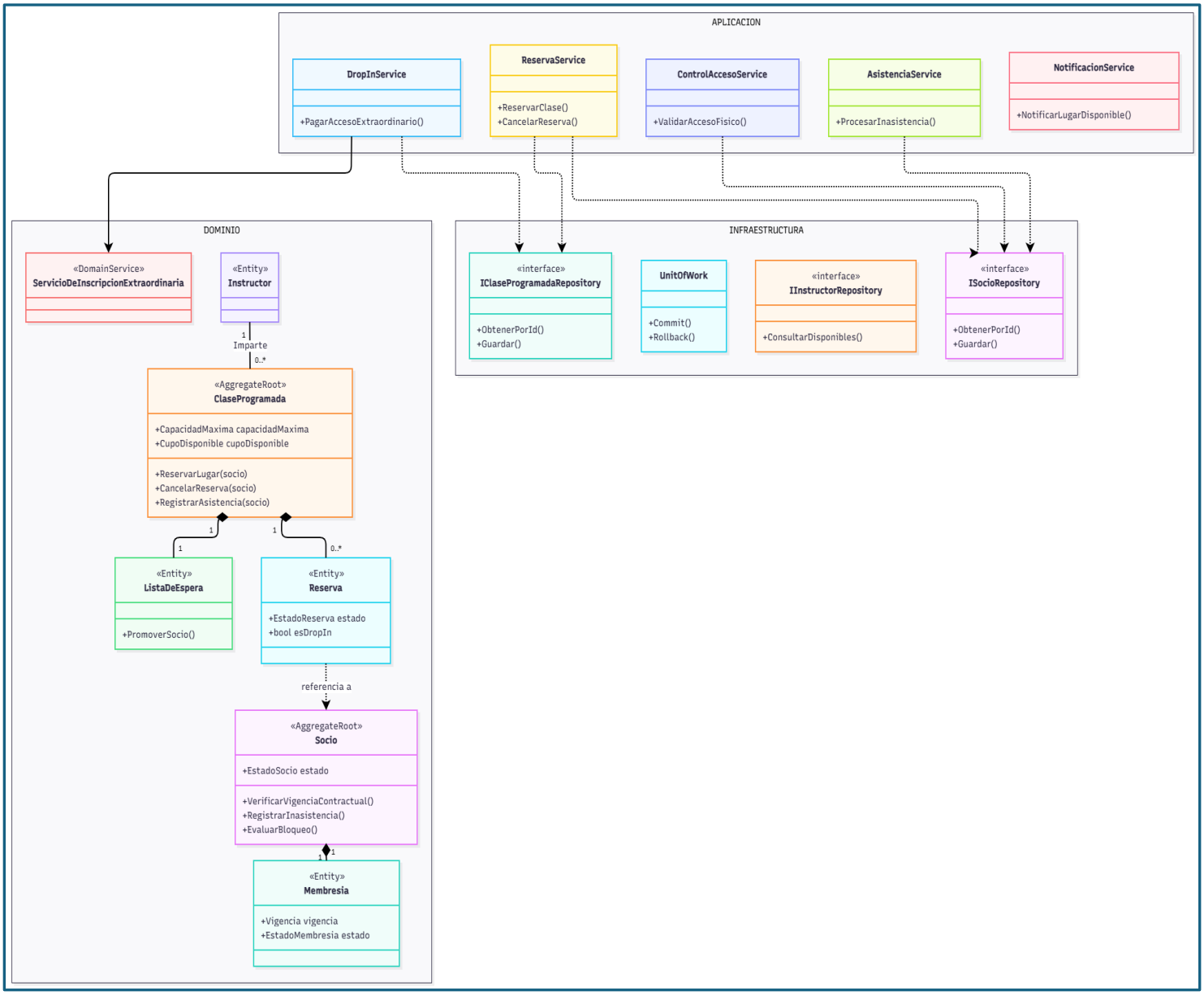
| # | Hallazgo | Evidencia | Impacto | Cambio realizado |
|---|---|---|---|---|
| 1 | ReservaService concentraba notificación y coordinación (riesgo S) | El método CancelarReserva enviaba el correo de notificación directamente dentro del mismo método | Dificultaría cambiar el canal de notificación sin tocar la coordinación de la cancelación | Se extrajo NotificacionService como colaborador aparte, invocado por evento LugarLiberadoEnClase |
| 2 | El umbral de no-shows podía quedar fijo en Socio (riesgo O) | Si el valor "3" se escribiera directamente en RegistrarInasistencia() | Cada cambio de política de sanciones obligaría a modificar la entidad Socio | Se aisló el umbral como parámetro de configuración leído por EvaluarBloqueo() |
| 3 | Se evaluó heredar ReservaDropIn de Reserva (riesgo L) | Un subtipo podría rechazar la operación Cancelar() con reglas distintas a una reserva normal | Rompería el contrato esperado de Reserva en el resto del sistema (ControlAccesoService, ReservaService) | Se usa el atributo esDropIn en lugar de un subtipo |
| 4 | Un borrador inicial proponía un repositorio único "IReservaYListaRepository" (riesgo I) | Mezclaba métodos de ClaseProgramada y de Reserva en una sola interfaz | ControlAccesoService dependería de métodos de ClaseProgramada que nunca usa | Se mantuvo separado: IClaseProgramadaRepository e ISocioRepository, cada uno con solo los métodos que su consumidor usa |
| 5 | Un boceto temprano llamaba directamente a Entity Framework desde AsistenciaService (riesgo D) | Código de ejemplo con DbContext inyectado en el Application Service | Cambiar de proveedor de base de datos rompería la capa de aplicación | Se introdujo ISocioRepository como abstracción; AsistenciaService ya no conoce el ORM |

## Etapa 8: Representar y documentar la arquitectura

Herramienta usada: Draw.io.

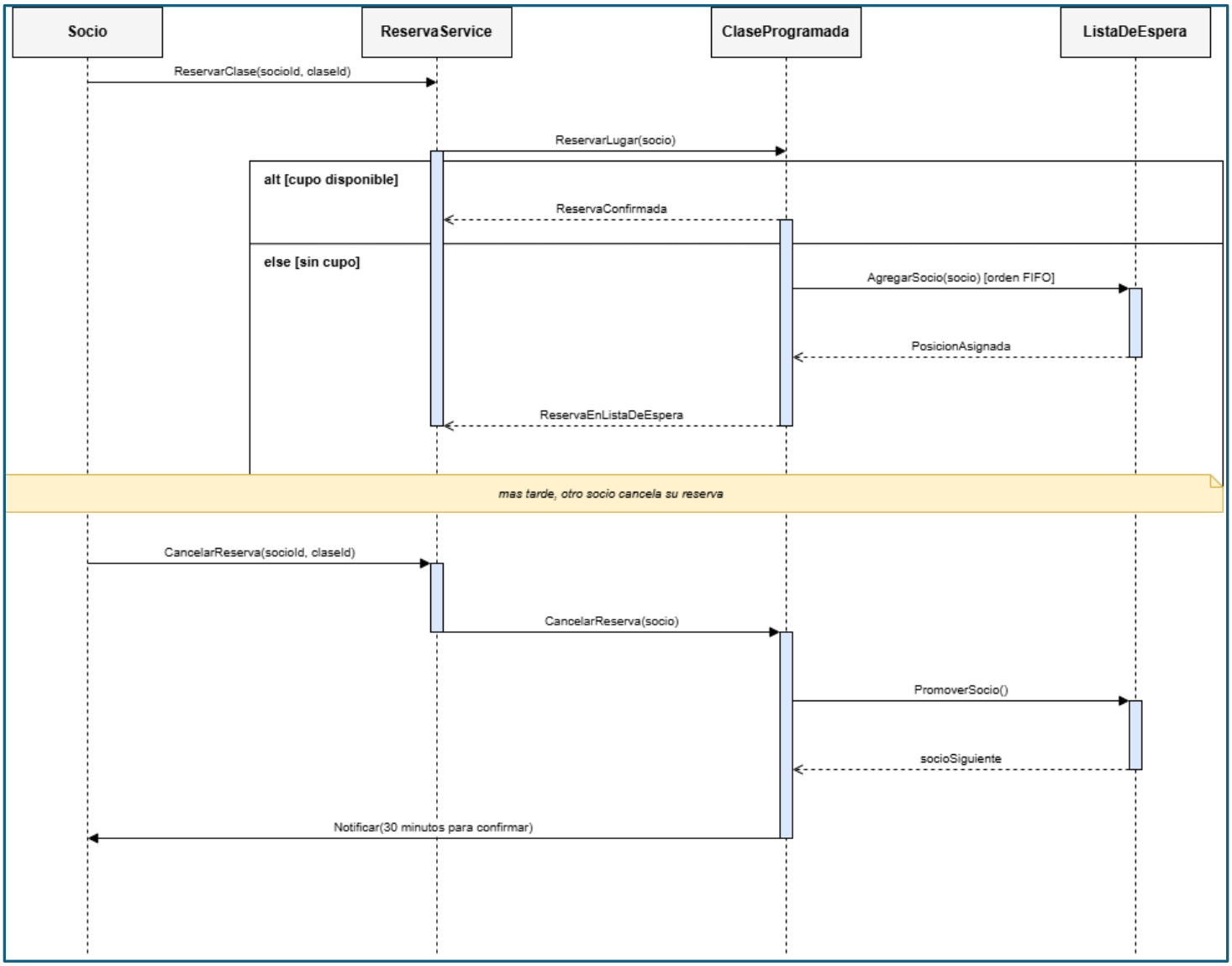
Los archivos fuente .drawio se incluyen en /diagrams del repositorio.

## Diagrama de clases



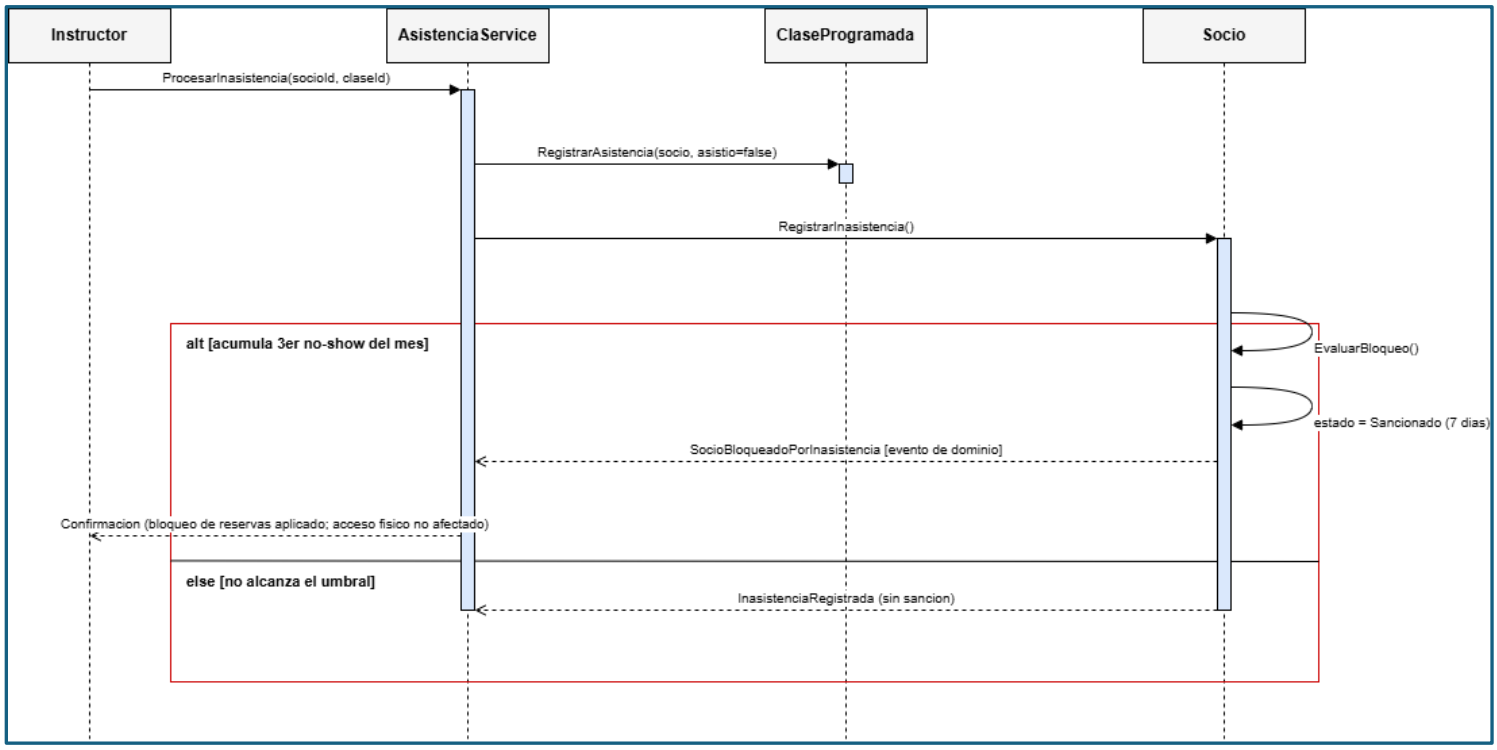
## Diagrama de secuencia 1 (flujo principal)

CU-01/CU-02: reservar clase, con desvío a lista de espera y liberación automática posterior:



## Diagrama de secuencia 2 (flujo de excepción)

CU-05: acumulación de no-shows con sanción automática:




## Architecture Decision Records

### ADR-01 — Adoptar Domain Model como patrón de lógica de negocio

| | |
|---|---|
| **ADR-01** | **Adoptar Domain Model como patrón de lógica de negocio** |
| **Estado** | Aceptada |
| **Contexto** | El proceso de reservación de FitZone tiene reglas numerosas e interdependientes con manejo de estado (membresía, cupo, lista de espera, no-shows), confirmadas en el BRD (N1-N4) y en US-001 a US-020. |
| **Alternativas** | Transaction Script (duplicaría reglas entre scripts, vulnerable a condiciones de carrera) y Table Module (no protege invariantes entre tablas relacionadas, dependencias circulares entre módulos). |
| **Decisión** | Se adopta Domain Model para todo el núcleo de reservación, membresías y asistencia; no se combina con otro patrón. |
| **Consecuencias** | Mayor inversión inicial en el mapa de dominio (Etapa 3); menor costo de mantenimiento al cambiar reglas de negocio; trabajo posterior: mantener los Aggregates pequeños al crecer el sistema. |
| **Trazabilidad** | N1-N4 (BRD); US-001, US-004, US-006, US-017 |

## ADR-02 — Usar Unit of Work para transacciones compuestas

| | |
|---|---|
| **ADR-02** | **Usar Unit of Work para transacciones compuestas** |
| **Estado** | Aceptada |
| **Contexto** | Cancelar una reserva y promover al siguiente socio de la lista de espera ocurren dentro del mismo Aggregate (ClasePrograma da) y deben confirmarse como una sola operación atómica. |
| **Alternativas** | Guardar los cambios de ClasePrograma da sin una transacción explícita (riesgo de que el lugar quede "volando" o se corrompa el aforo si falla a mitad de camino). |
| **Decisión** | Se introduce un Unit of Work que inicia al comenzar el Application Service y hace Commit solo cuando ClasePrograma da guarda el nuevo estado con el socio promovido; ante error, Rollback completo. |
| **Consecuencias** | Mayor consistencia de datos; una capa adicional de infraestructura a mantener; trabajo posterior: definir el manejo de reintentos ante fallos de red. |
| **Trazabilidad** | US-004, US-005; CU-03 |

## ADR-03 — Aplicar CQRS básico solo en consultas de alto volumen

| | |
|---|---|
| **ADR-03** | **Aplicar CQRS básico solo en consultas de alto volumen** |
| **Estado** | Aceptada |
| **Contexto** | ConsultarDisponibilidadClasesQuery y ConsultarEstadoSocioQuery son de solo lectura y de alto volumen (consultadas por cientos de usuarios desde la app), distintas en naturaleza a los Commands de reserva/cancelación/sanción. |
| **Alternativas** | Aplicar CQRS a todo el sistema (mayor complejidad sin beneficio claro) o no aplicarlo en ningún lado (consultas lentas si cargan el Aggregate ClasePrograma da completo en cada lectura). |
| **Decisión** | Se separan Queries de Commands únicamente para las consultas de alto volumen; ValidarAccesoFisico (CU-04) se mantiene como método directo del servicio por ser una lectura puntual de bajo costo. |
| **Consecuencias** | Consultas más rápidas y simples de construir; dos rutas de código a mantener solo donde se justifica; trabajo posterior: evaluar si los reportes administrativos de ocupación (US-014) también deben pasar a Query. |
| **Trazabilidad** | US-002, US-006; N1, N3 |

## Herramienta y contexto

| Aspecto | Detalle |
|---|---|
| Herramienta | Claude (Anthropic), en una sesión de trabajo con el BRD, las User Stories y la guía oficial de la Unidad 3 adjuntos. |
| Insumos entregados a la IA | BRD y User Stories de FitZone (U2), guía oficial de la Unidad 3 (Actividad_Unidad_3_CLN_ArquitecturaAlumnos.pdf). |
| Datos excluidos | No se envió información ajena al caso académico de FitZone. |
| Salidas de la IA | Informe de arquitectura (.docx, 13 páginas), prompts/XML de draw.io para el diagrama de clases y los dos diagramas de secuencia, código base en Python (dominio, aplicación, infraestructura). |
| Responsabilidad | El equipo revisó el informe y lo aprobó con observaciones de formato (reducción de preguntas de entrevista, incorporación de ideas del borrador de un compañero). Cada integrante debe poder defender las decisiones sin leer una respuesta generada. |

## Ciclo de trabajo aplicado

| Paso | Acción | Evidencia |
|---|---|---|
| 1 | Preparar contexto | Se seleccionaron fragmentos relevantes del BRD, las User Stories, el BPMN y el glosario oficiales del Equipo 11. |
| 2 | Solicitar propuesta | Se pidió una alternativa concreta a la IA, exigiendo que declarara sus supuestos. |
| 3 | Comparar | Se contrastó la respuesta contra el caso FitZone y los conceptos DDD de la unidad. |
| 4 | Corregir | Se modificaron nombres, responsabilidades, límites y dependencias que no correspondían. |
| 5 | Validar | Se revisó trazabilidad, consistencia entre diagramas y comportamiento esperado (Etapa 9). |
| 6 | Registrar | Se guardó prompt, respuesta, decisión humana y versión resultante en la bitácora siguiente. |

## Bitácora de iteraciones

Las iteraciones 1 a 4 corresponden a la bitácora oficial incluida en el informe de arquitectura (Etapa 8). Las iteraciones 5 y 6 documentan trabajo posterior con IA, ya con el informe validado: la corrección de los diagramas UML en draw.io y la generación del código base (entregable 06).

| It. | Objetivo | Prompt (resumen) | Respuesta útil | Error o supuesto detectado | Decisión del equipo |
|---|---|---|---|---|---|
| 1 | Elegir el patrón de lógica de negocio | Comparar Transaction Script, Table Module y Domain Model para FitZone con base en las reglas del BRD de U2, exigiendo declarar supuestos. | Tabla comparativa de 4 criterios + justificación por alternativa. | La IA inicialmente sugería una combinación de patrones sin delimitar dónde se usaría cada uno. | Se adoptó Domain Model puro (ADR-01), sin combinación de patrones. |
| 2 | Construir el mapa de dominio | Proponer candidatos a Entidades, Value Objects, Aggregates y Domain Events a partir del glosario y las reglas de U2, indicando dudas para validación humana. | Mapa de dominio con 2 Aggregates (ClaseProgramada, Socio) y su diagrama visual. | La IA proponía inicialmente Reserva como Aggregate independiente, ignorando el riesgo de condiciones de carrera. | Se descartó Reserva como Aggregate independiente (Etapa 3.3). |
| 3 | Diseñar la Service Layer | Proponer firmas de Application Services a partir de los 5 casos de uso, separando coordinación de reglas de dominio. | Tabla de 5 servicios/métodos + checklist de revisión obligatoria respondido. | Ninguno relevante; se verificó que ningún método contuviera lógica de negocio directamente. | Se aceptaron los 5 métodos propuestos, ajustando solo nombres al glosario del equipo. |
| 4 | Revisar el diseño con SOLID | Señalar posibles violaciones SOLID en la Service Layer, Repository y Aggregates, sin reescribir todo el diseño. | 5 hallazgos con evidencia, impacto y cambio sugerido por principio. | La IA sugería heredar ReservaDropIn de Reserva; riesgo de violar Liskov detectado por el equipo. | Se usó un atributo esDropIn en vez de herencia (hallazgo 3, Etapa 7). |
| 5 | Generar los diagramas UML en draw.io | Prompt de texto natural para el diagrama de clases y los 2 diagramas de secuencia, con instrucciones explícitas para evitar un layout tipo BPMN. | 3 prompts de draw.io en lenguaje natural, uno por diagrama. | El generador de IA de draw.io producía texto encimado dentro de las cajas y un layout tipo BPMN, incluso tras instrucciones más explícitas. | Se reemplazaron los prompts en lenguaje natural por XML exacto de draw.io (mxGraph), generado y validado con un script, eliminando el problema de encimado. |
| 6 | Generar el código base (entregable 06) | Pedir una implementación boilerplate en Python (dominio, aplicación, infraestructura) a partir del informe de arquitectura ya validado. | Paquete con 2 Aggregates, 5 Application Services, repositorios en memoria, Unit of Work y 9 pruebas automáticas. | El límite de reservas simultáneas por plan se calculaba contra el total de reservas de la clase, en vez de contra las reservas del propio socio. | Se corrigió el cálculo para contar las reservas activas del socio a través de todas las clases, antes de integrar el código al entregable. |

## Verificaciones técnicas realizadas

| Verificación | Cómo se hizo | Resultado |
|---|---|---|
| Validez del .docx | Conversión a PDF (LibreOffice) y revisión visual página por página antes de entregar. | Validación aprobada, sin errores de formato. |
| Sintaxis de los diagramas | Validación del XML de draw.io (mxGraph) con un parser antes de entregarlo. | Sin errores; diagramas renderizados sin texto encimado. |
| Consistencia entre diagramas | Comparación de los participantes y métodos de los 2 diagramas de secuencia contra las clases del diagrama de clases. | Todos los nombres coinciden. |
| Comportamiento esperado | 9 pruebas automáticas (pytest) sobre los 5 casos de uso del código base. | 9 de 9 aprobadas. |

## Validación humana de los aportes de la IA

| Aporte de la IA | Contraste con la Unidad 2 | Resultado |
|---|---|---|
| Domain Model como patrón | Las reglas del BRD (N1-N4) y US-001/US-013 exigen proteger invariantes de cupo y vigencia. | Aceptado (ADR-01) |
| Reserva como Aggregate independiente | Contradice el límite de consistencia necesario para evitar condiciones de carrera al reservar. | Rechazado |
| Herencia de ReservaDropIn sobre Reserva | Rompería el contrato esperado de Reserva en ControlAccesoService y ReservaService (riesgo Liskov). | Rechazado — se usó un atributo esDropIn |
| Umbral de no-shows fijo dentro de Socio | Cada cambio de política de sanciones obligaría a modificar la entidad (riesgo Open/Closed). | Corregido — se aisló como parámetro configurable |
| Repositorio único para Socio y ClaseProgramada | ControlAccesoService dependería de métodos de ClaseProgramada que nunca usa (violación de I). | Rechazado — se separaron ISocioRepository e IClaseProgramadaRepository |
| Límite de reservas simultáneas contado por clase | No corresponde al negocio: el límite es por socio (US-001), no por clase. | Corregido en el código base antes de integrarlo |

## Errores detectados y correcciones

- **Diagrama de clases:** El generador de IA de draw.io producía un diagrama de clases con texto encimado y layout tipo BPMN en los dos primeros intentos.
- **Código base:** El código base calculaba el límite de reservas simultáneas por plan contra el total de la clase, no contra las reservas del propio socio.
- **Formato de diagramas:** La primera versión de los diagramas usaba PlantUML; se corrigió a draw.io por preferencia explícita del equipo.

## Propuestas de la IA rechazadas o corregidas (pregunta de defensa)

Reserva como Aggregate independiente, un Aggregate financiero de "saldos a favor" para reembolsos, herencia de ReservaDropIn sobre Reserva, y los dos primeros diagramas de clases generados automáticamente por el asistente de IA de draw.io sin control de layout.

## Prompts utilizados

### 7.1 Mensaje 1 (enviado por el equipo)

Realiza un boceto de la siguiente actividad, en base a la actividad anterior, donde analices detalladamente cada uno de los apartados para crear una versión decente que posteriormente analizaremos, puedes utilizar cualquier formato, al fin de cuentas tu aporte es solo un boceto de lo que será la actividad completa que nosotros crearemos.

*[Adjuntando: Todos los documentos de la Actividad 2]*

*[Adjuntando: Actividad_Unidad_3_CLN_ArquitecturaAlumnos.pdf]*

## Formatos de prompt de la guía aplicados al caso (reproducibles)

Son los formatos de prompt sugeridos por la guía oficial, completados con los datos de FitZone. Permiten que el equipo los reproduzca y compare resultados.

| Tema | Prompt |
|---|---|
| Patrones | Con base en las reglas de cut-off, FIFO y bloqueo del BRD de FitZone (N1-N4) y los casos de uso CU-01 a CU-05, compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias. |
| Dominio | Revisa este glosario y estas reglas de FitZone [Socio, Membresía, ClaseProgramada, Reserva, ListaDeEspera, Instructor]. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Para cada candidato, indica la evidencia utilizada y las dudas que requieren validación humana. |
| Service Layer | A partir de los casos de uso CU-01 a CU-05 de FitZone, propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa. |
| SOLID | Analiza este diseño de FitZone [Service Layer, Repository, Aggregates] y señala posibles violaciones SOLID. No reescribas todo; identifica evidencia, riesgo y alternativas para que el equipo decida. |
| Diagramas | Convierte esta arquitectura validada de FitZone en un diagrama de draw.io (prompt de texto o XML mxGraph). No agregues clases, reglas o relaciones que no aparezcan en la descripción. |

## Declaración de uso responsable y pendientes

- Cada aporte de la IA fue revisado contra el BRD y las User Stories antes de integrarse.
- Los supuestos y decisiones dudosas quedaron documentados en la sección 3.3 del informe ("Decisiones dudosas o descartadas").
- Pendiente del equipo: subir el código base y los archivos fuente de los diagramas (.drawio) al repositorio de GitHub.

- Pendiente del equipo: cada integrante debe ensayar la defensa de las decisiones sin apoyo de la IA.

## Etapa 9: Validar la propuesta

| Validación | Pregunta | Resultado del equipo |
|---|---|---|
| Trazabilidad | ¿Cada componente importante responde a una necesidad, regla o caso de uso? | Sí — los 5 Application Services, los 2 Aggregates y el Domain Service se relacionan con al menos una US o necesidad N1-N4 (ver matriz abajo). |
| Lenguaje | ¿Los mismos términos significan lo mismo en BRD, UML, código y ADR? | Sí — se reutilizaron los términos del glosario oficial (Socio, Membresía, ClaseProgramada, Reserva, ListaDeEspera, Vigencia) sin sinónimos nuevos. |
| Responsabilidad | ¿Las reglas permanecen en el dominio o en el patrón elegido con justificación? | Sí — verificado en la Revisión obligatoria de la Etapa 4; ninguna regla de negocio vive en un Application Service. |
| Consistencia | ¿Los diagramas de clase y secuencia representan la misma arquitectura? | Sí — los participantes de ambos diagramas de secuencia (ReservaService, ClaseProgramada, ListaDeEspera, AsistenciaService, Socio) corresponden a clases del diagrama de clases. |
| Persistencia | ¿El dominio evita depender directamente de SQL, ORM o infraestructura? | Sí — ClaseProgramada, Reserva, Socio y Membresia no referencian ningún repositorio ni ORM; solo los Application Services dependen de las interfaces (Etapa 5). |
| Transacción | ¿El límite de la operación y su manejo de error están definidos? | Sí — ver Unit of Work (Etapa 5): CancelarReserva define dónde inicia, qué incluye (liberar + promover) y qué pasa ante un error (Rollback completo). |
| IA | ¿Cada aporte de IA fue revisado y su uso está registrado? | Sí — bitácora de 4 iteraciones con error/supuesto detectado y decisión del equipo en cada una. |

## Matriz de trazabilidad

| NECESIDAD DE NEGOCIO / PROBLEMA | CASO DE USO | REGLAS DE NEGOCIO RELACIONADAS | HISTORIAS DE USUARIO (US) | COMPONENTES DE ARQUITECTURA | DECISIONES (ADR) |
|---|---|---|---|---|---|
| N2 / N3: Fuga de capacidad por reservas sin control de cupo ni vigencia. | CU-01 (Reservar clase) | N2 (límite de reservas por plan) N3 (cupo > 0, vigencia activa) | US-001, US-002, US-013 | Dominio: ClaseProgramada, Reserva, Socio, Membresía. Aplicación: ReservaService.ReservarClase. Eventos: ReservaConfirmada. | ADR-01 (Domain Model) |
| N3: Pérdida de la oportunidad de asistir al agotarse el cupo sin un mecanismo de espera. | CU-02 (Lista de espera) | N3 (cupo == 0, orden FIFO, ventana de 30 min para confirmar) | US-003, US-005 | Dominio: ListaDeEspera, ClaseProgramada.ReservarLugar. Aplicación: ReservaService.ReservarClase (delega a lista de espera). | ADR-01 (Domain Model) |
| N2 / N3: Cupo retenido sin reasignación automática cuando un socio cancela. | CU-03 (Cancelar reserva) | N2 (cancelación con anticipación) N3 (reasignación FIFO) | US-004, US-005 | Dominio: ClaseProgramada.CancelarReserva, ListaDeEspera.PromoverSocio. Aplicación: ReservaService.CancelarReserva. Eventos: LugarLiberadoEnClase. | ADR-01 (Domain Model) ADR-02 (Unit of Work) |

| NECESIDAD DE NEGOCIO / PROBLEMA | CASO DE USO | REGLAS DE NEGOCIO RELACIONADAS | HISTORIAS DE USUARIO (US) | COMPONENTES DE ARQUITECTURA | DECISIONES (ADR) |
|---|---|---|---|---|---|
| N1: Acceso físico sin validar vigencia de membresía de forma automática. | CU-04 (Validar acceso) | N1 (verificación automática, motivo explícito de rechazo) | US-006, US-007, US-018 | Dominio: Socio.VerificarVigenciaContractual, Membresía. Aplicación: ControlAccesoService.ValidarAccesoFisico. Eventos: AccesoFisicoDenegado, MembresiaVencida. | ADR-01 (Domain Model) ADR-03 (CQRS básico — se mantiene como método directo) |
| N4: Acumulación de inasistencias sin una política de sanción automática y consistente. | CU-05 (Sanción por no-shows) | N4 (3 no-shows/mes → bloqueo de 7 días en reservas; acceso físico no afectado) | US-011, US-015, US-017 | Dominio: Socio.RegistrarInasistencia, Socio.EvaluarBloqueo. Aplicación: AsistenciaService.ProcesarInasistencia. Eventos: SocioBloqueadoPorInasistencia. | ADR-01 (Domain Model) |

## Conclusión

El proceso de diseño arquitectural confirmó que Domain Model era la opción correcta para FitZone: las reglas de negocio del caso (cupo máximo, orden FIFO, ventana de confirmación, bloqueo por inasistencias) están fuertemente interrelacionadas y dependen de mantener invariantes de estado, algo que Transaction Script y Table Module no podían garantizar sin duplicar lógica o exponer condiciones de carrera.

Separar la Service Layer, los Repository/Unit of Work y los Commands/Queries permitió aislar completamente las reglas de negocio dentro del dominio: ningún Application Service decide, todos delegan en ClasePrograma o en Socio. La revisión con SOLID encontró 5 violaciones reales durante el proceso (no hipotéticas) y las 5 se corrigieron antes de cerrar el diseño, lo que se documentó en la bitácora de IA junto con las propuestas que el equipo rechazó.

El código base en Python valida que el diseño funciona: las 9 pruebas automáticas cubren los 5 casos de uso, incluyendo un caso que detectó un error real de implementación (el límite de reservas simultáneas se contaba mal) y que se corrigió antes de integrarlo al entregable. Con esto, el equipo considera que la arquitectura propuesta está lista para pasar a una siguiente fase de implementación completa.

## Conclusiones individuales

### Conclusión de Christopher Cortez Garcia

Esta actividad me dejó claro que diseñar la arquitectura antes de programar no es un paso burocrático, sino lo que evita que el sistema se vuelva imposible de mantener. Trabajar el caso de FitZone me hizo entender por qué un Domain Model era la única opción viable: reglas como el cupo máximo, la lista de espera FIFO y el bloqueo por inasistencias no se pueden proteger bien si se dejan sueltas en scripts o repartidas entre tablas. Lo que más trabajo me costó fue aceptar que una propuesta "más simple" (como independizar Reserva o usar herencia en ReservaDropIn) en realidad

escondía riesgos reales de inconsistencia, y que rechazarlas con una justificación sólida era parte del ejercicio, no un paso de más. Usar IA como apoyo aceleró el borrador de cada etapa, pero el trabajo real estuvo en contrastar cada propuesta contra el BRD y las User Stories, corregir lo que no aplicaba a FitZone, y al final verificarlo con código funcional: las 9 pruebas en verde confirmaron que la arquitectura que documentamos realmente se sostiene, no solo se ve bien en el papel. Me queda la responsabilidad de poder defender estas decisiones sin apoyarme en el documento, porque entenderlas fue justamente el objetivo de la actividad.

**Conclusión de Oscar Elias Vera Moreno**

Esta actividad me permitió entender que diseñar la arquitectura de un sistema no consiste en

empezar a programar, sino en decidir primero dónde vive cada regla del negocio y por qué. Al trabajar con el caso FitZone pasé de los requerimientos de la Unidad 2 a una propuesta que separa dominio, aplicación e infraestructura, y comprendí que esa separación es lo que hace que el sistema sea mantenible cuando las reglas cambian, por ejemplo, si el umbral de no-shows pasara de 3 a 4.

Lo que más aprendí fue la importancia de justificar las decisiones con evidencia. Elegir el Domain Model frente a Transaction Script y Table Module no fue una preferencia, sino una consecuencia de reglas concretas como el límite de reservas por plan, la lista de espera FIFO con ventana de 30 minutos y la sanción de 7 días por tres inasistencias. También me quedó claro por qué ClasePrograma debe ser la raíz del agregado que protege el cupo: de otro modo, la sobreventa por condiciones de carrera sería posible. Las decisiones descartadas, como tratar Reserva como agregado independiente o modelar saldos a favor, me enseñaron que saber qué no incluir es parte del diseño.

Las etapas de Service Layer, Repository, Unit of Work y CQRS me ayudaron a ver cómo cada patrón resuelve un problema específico: los servicios de aplicación solo coordinan, los repositorios aíslan la base de datos, el Unit of Work garantiza que liberar un lugar y promover al siguiente socio ocurran de forma atómica, y CQRS se justifica únicamente en las consultas de alto volumen. La revisión con SOLID, con sus cinco hallazgos, mostró que incluso un diseño que parece correcto puede esconder acoplamientos que conviene corregir antes de escribir código.

Sobre el uso de la IA, la bitácora dejó una lección importante: la herramienta propone rápido, pero también se equivoca. Detectamos y corregimos errores como la herencia de ReservaDropIn, la combinación de patrones sin delimitar y el cálculo del límite de reservas por clase en lugar de por socio. Esto confirma que la IA es un apoyo y que la responsabilidad de validar contra el caso real es del equipo.

Como pendientes personales, me queda ensayar la defensa de las decisiones sin apoyo de material generado y colaborar en subir el código base y los archivos .drawio al repositorio. En conjunto, la

actividad fortaleció mi capacidad para analizar un problema de negocio, traducirlo a un modelo de dominio y defender cada decisión arquitectónica con argumentos, algo que considero esencial para mi formación como ingeniero de software.

**Conclusión de Christofer Aaron Torres Puga**

En conclusión, el desarrollo de este proyecto me permitió transformar los requerimientos iniciales de FitZone en una arquitectura de software robusta y lista para escalar. Considero que la decisión más importante que tomamos como equipo fue implementar el Domain Model; después de analizar otras opciones, me quedó claro que era la única forma de proteger reglas tan estrictas como el límite de cupos y resolver el problema de los "no-shows" sin enredar el código.

Para lograrlo, aplicamos conceptos de Arquitectura Limpia y DDD, definiendo muy bien los agregados principales y usando herramientas como Unit of Work para que las operaciones complejas (como cancelar y mover listas de espera) no se queden a medias. También pensamos en el rendimiento integrando un CQRS básico, lo que asegura que el sistema aguante muchas consultas a la vez sin saturar la base de datos.

Por último, esta actividad me dejó una gran lección sobre el uso de la Inteligencia Artificial. Me sirvió muchísimo como apoyo, pero aprendí que no se le puede aceptar todo a ciegas. Tuvimos que ser críticos, corregirle errores que rompían los principios SOLID y rechazar ideas que habrían causado problemas graves de concurrencia. Al final, logramos entregar un diseño justificado, donde cada componente tiene una razón de ser y responde directamente a lo que el gimnasio realmente necesita.

## Link del Repositorio:

https://github.com/cortezchristopher704-rgb/FitZone

## Estructura del repositorio de GitHub

El repositorio sigue la estructura sugerida por la guía oficial (sección 16): un README general, y carpetas separadas para documentación, diagramas, código fuente y pruebas.

```
FitZone/
├── diagrams/
│   ├── DIAGRAMA DE CLASES.pdf / .png
│   ├── DIAGRAMA DE DOMINIO.pdf / .jpeg
│   ├── DIAGRAMA DE SECUENCIA 1.png
│   ├── DIAGRAMA DE SECUENCIA 2.png
│   ├── Diagrama de secuencia 1 (flujo principal).pdf
│   └── Diagrama de secuencia 2 (flujo de excepción).pdf
├── docs/
│   ├── ACTIVIDAD 3 COMPONENTES DE LÓGICA NEGOCIO.docx
│   ├── ACTIVIDAD 3 REPORTE COMPONENTES DE LÓGICA NEGOCIO.pdf
│   ├── BITACORA DE IA.docx / .pdf
│   └── Matriz de Trazabilidad.docx / .pdf
├── src/
│   ├── aplicacion/         Application Services + CQRS (Etapas 4 y 6)
│   ├── dominio/            Aggregates, Entities, Value Objects, Domain Events (Etapa 3)
│   └── infraestructura/    Repository + Unit of Work (Etapa 5)
├── tests/                  9 pruebas automáticas sobre los 5 casos de uso
├── README.md
├── pytest.ini
└── requirements.txt
```

Cada carpeta corresponde directamente a un entregable: docs/ cubre los entregables 01, 04 y 05 (informe, matriz y bitácora, en Word y PDF); diagrams/ cubre el 02 y el 03 (diagrama de dominio, de clases y los 2 de secuencia, cada uno en PDF y en imagen); y src/ + tests/ cubren el entregable 06 (código base dividido en aplicación, dominio e infraestructura, con 9 pruebas en verde).

## Checklist final de entrega

Verificación de los 8 entregables oficiales de la actividad antes de subir el reporte.

| # | Entregable | Estado | Ubicación |
|---|---|---|---|
| 01 | Informe de arquitectura (Etapas 1-9) | Completo | Este documento |
| 02 | Diagrama de dominio y clases | Completo | Etapa 3 y 8 del informe + /diagrams/ |
| 03 | Diagramas de secuencia (2) | Completo | Etapa 8 del informe + /diagrams/ |
| 04 | Matriz de trazabilidad | Completo | Etapa 9 del informe |
| 05 | Bitácora de IA | Completo | Sección de Herramienta y contexto / Bitácora de iteraciones |
| 06 | Código base | Completo | /src, /tests del repositorio (9 pruebas) |
| 07 | Repositorio de GitHub | Completo | https://github.com/cortezchristopher704-rgb/FitZone |
| 08 | Presentación y defensa | Pendiente de ensayo | Ver 'Preguntas para la defensa' abajo |

## Preguntas para la defensa

Preguntas literales de la sección 20 de la guía oficial, respondidas con el diseño real de FitZone.

**1. ¿Qué evidencia del negocio influyó más en la selección del patrón?**

El BRD (N1-N4) y las User Stories US-001/US-013, que exigen proteger invariantes muy condicionales: cupo máximo, orden FIFO con ventana de 30 minutos, y bloqueo por 3 no-shows sin afectar el acceso físico. Esas reglas cruzan varias entidades a la vez, lo que descartó Transaction Script y Table Module y llevó a Domain Model (ADR-01).

**2. ¿Qué responsabilidad fue la más difícil de ubicar y cómo tomaron la decisión?**

Decidir si Reserva debía ser un Aggregate independiente o una entidad dentro de ClasePrograma da. Se descartó independizarla porque permitir que la aplicación la insertara directamente generaría condiciones de carrera sobre el cupo; se dejó como entidad contenida en ClasePrograma da, que es la única puerta para reservar (sección 3.3 del informe, 'Decisiones dudosas y descartadas').

**3. ¿Qué regla protege el diseño y qué ocurriría si estuviera en una capa incorrecta?**

La invariante de que las reservas activas nunca superan la CapacidadMaxima vive dentro de ClasePrograma da (dominio). Si estuviera en un Application Service o en la capa de infraestructura,

cualquier otro flujo podría saltársela y sobreasignar lugares, porque dejaría de ser una regla garantizada por el objeto y pasaría a depender de que cada desarrollador la revise manualmente.

**4. ¿Qué cambió después de la revisión con SOLID?**

Se corrigieron 5 hallazgos reales (Etapa 7): se separó NotificacionService de ReservaService (S), se aisló el umbral de no-shows como parámetro configurable (O), se evitó heredar ReservaDropIn de Reserva usando un atributo esDropIn (L), se dividió un repositorio único en ISocioRepository e IClaseProgramadaRepository (I), y los Application Services quedaron dependiendo de abstracciones en vez de un ORM concreto (D).

**5. ¿Qué propuesta de la IA rechazaron o corrigieron y por qué?**

Se rechazó modelar Reserva como Aggregate independiente (riesgo de condiciones de carrera) y un Aggregate financiero de "saldos a favor" para reembolsos (las clases son un servicio incluido en la suscripción, no admiten devoluciones unitarias según el BRD). También se corrigió un error real en el código base generado: el límite de reservas simultáneas se contaba contra el total de la clase en vez de contra las reservas del propio socio.

**6. ¿Qué consecuencia negativa aceptaron al tomar su decisión arquitectural principal?**

Adoptar Domain Model de forma irrestricta implicó una mayor inversión inicial de diseño: hubo que identificar correctamente los Aggregates y sus límites de consistencia (Etapa 3) antes de poder escribir cualquier Service Layer, en vez de empezar a programar directamente como permitiría Transaction Script. El equipo aceptó ese costo a cambio de mayor solidez ante las reglas de excepción del caso.